

Homework 08 - Java Multithreading

Topics Covered

- Thread
- Multithreading
- Process vs Thread
- Main thread
- Creating thread using `Thread` class
- Creating thread using `Runnable` interface
- `start()` vs `run()`
- Thread names
- `sleep()`
- `join()`
- Thread priority
- Daemon thread
- Thread lifecycle
- Race condition
- `synchronized`
- Synchronized block
- `Lock`
- `ReentrantLock`
- `tryLock()`
- `wait()` and `notify()`
- Executor Framework
- `ExecutorService`
- `FixedThreadPool`
- `SingleThreadExecutor`
- `CachedThreadPool`
- `ScheduledExecutorService`
- `execute()` vs `submit()`
- `Callable`
- `Future`
- `invokeAll()`
- `invokeAny()`

Instructions

1. Write every program in a separate Java file.
2. First write the expected output, then run the program.
3. If output order can change, write: `Output order is not guaranteed`.
4. For every program, write which thread is doing the work.
5. For synchronization/lock questions, explain what shared data is being protected.

6. For `ExecutorService` programs, always call `shutdown()` unless task is intentionally repeated.
7. After every coding question, write 2-3 lines explaining real-time use.

Part A: Short Answer Questions

1. What is a thread?
2. What is multithreading?
3. What is the difference between process and thread?
4. Why do backend applications need multithreading?
5. What is the main thread?
6. How can we get the current thread name?
7. What are two basic ways to create a thread in Java?
8. What is the use of `run()` method?
9. What is the use of `start()` method?
10. What is the difference between `start()` and `run()` ?
11. Can we start the same thread object twice?
12. What is the use of `sleep()` ?
13. Why does `sleep()` need exception handling?
14. What is the use of `join()` ?
15. Where can we use `join()` in a real project?
16. Does thread priority guarantee execution order?
17. What is a daemon thread?
18. What is a race condition?
19. Why does race condition happen?
20. What is the use of `synchronized` ?
21. What is the difference between synchronized method and synchronized block?
22. What is `ReentrantLock` ?
23. Why do we write `unlock()` inside `finally` ?
24. What is the use of `tryLock()` ?
25. What is the difference between `synchronized` and `Lock` ?
26. What is Executor Framework?
27. Why is `ExecutorService` better than creating raw threads again and again?
28. What is `FixedThreadPool`?
29. What is `SingleThreadExecutor`?
30. What is `ScheduledExecutorService`?
31. What is the difference between `execute()` and `submit()` ?
32. What is `Callable` ?
33. What is `Future` ?
34. What is the use of `future.get()` ?
35. What is the difference between `invokeAll()` and `invokeAny()` ?

Part B: Find The Output

Question 1

```
public class Main {
    public static void main(String[] args) {
        System.out.println(Thread.currentThread().getName());
    }
}
```

Question 2

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Running in " + Thread.currentThread().getName());
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();

        t1.run();
        t1.start();
    }
}
```

Write:

Which line runs in main thread?
Which line runs in new thread?

Question 3

```
class NumberThread extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(getName() + " " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        NumberThread t1 = new NumberThread();
        NumberThread t2 = new NumberThread();

        t1.setName("Thread-A");
        t2.setName("Thread-B");

        t1.start();
        t2.start();
    }
}
```

Write:

Is exact output order guaranteed?

Question 4

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();

        t1.start();
        t1.start();
    }
}
```

Write:

What exception will occur and why?

Question 5

```
class DownloadTask extends Thread {
    public void run() {
        System.out.println("Downloading file");
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        DownloadTask task = new DownloadTask();
        task.start();
        task.join();

        System.out.println("Download completed, now processing");
    }
}
```

Question 6

```

class Counter {
    int count = 0;

    void increment() {
        count++;
    }
}

class CounterTask extends Thread {
    Counter counter;

    CounterTask(Counter counter) {
        this.counter = counter;
    }

    public void run() {
        for (int i = 1; i <= 1000; i++) {
            counter.increment();
        }
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();

        CounterTask t1 = new CounterTask(counter);
        CounterTask t2 = new CounterTask(counter);

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(counter.count);
    }
}

```

Write:

Expected value is 2000.
Can actual output be less than 2000? Why?

Question 7

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        for (int i = 1; i <= 4; i++) {
            int taskNumber = i;
            executor.submit(() -> {
                System.out.println("Task " + taskNumber + " by "
                    + Thread.currentThread().getName());
            });
        }

        executor.shutdown();
    }
}
```

Write:

How many worker threads are in the pool?
Is task order guaranteed?

Question 8

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class Main {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newSingleThreadExecutor();

        Callable<Integer> task = () -> 10 + 20;

        Future<Integer> future = executor.submit(task);

        System.out.println(future.get());

        executor.shutdown();
    }
}
```

Part C: Find And Fix The Problem

Question 1

```

class MyThread extends Thread {
    public void run() {
        System.out.println("Running");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.run();
    }
}

```

Write:

1. Is this multithreading?
2. What is the mistake?
3. Correct the code.

Question 2

```

class MyThread extends Thread {
    public void run() {
        System.out.println("Running");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
        t1.start();
    }
}

```

Write:

1. What exception occurs?
2. Why does it happen?
3. Correct the code.

Question 3

```

class Counter {
    int count = 0;

    void increment() {
        count++;
    }
}

```

Two threads are using same `Counter` object.

Write:

1. What problem can happen?
2. Correct using synchronized method.
3. Correct using synchronized block.

Question 4

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class Counter {
    int count = 0;
    Lock lock = new ReentrantLock();

    void increment() {
        lock.lock();
        count++;
    }
}
```

Write:

1. What is wrong?
2. Why can this block other threads?
3. Correct using `try-finally`.

Question 5

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        executor.submit(() -> System.out.println("Task completed"));
    }
}
```

Write:

1. What is missing?
2. Why can program keep running?
3. Correct the code.

Question 6

```
Future<String> f1 = executor.submit(task1);
System.out.println(f1.get());

Future<String> f2 = executor.submit(task2);
System.out.println(f2.get());
```


Write:

1. What is the design problem?
2. Why does it reduce parallel benefit?
3. Rewrite better approach.

Part D: Coding Practice

Program 1: Create Thread Using Thread Class

Create class `NumberThread` that extends `Thread` .

It should print numbers from 1 to 5.

Create two thread objects and start both.

Write:

```
Is output order fixed?
```

Program 2: Create Thread Using Runnable

Create class `EmailTask` that implements `Runnable` .

Inside `run()` , print:

```
Sending email by <thread-name>
```

Create 3 threads using same `EmailTask` style and start them.

Program 3: `start()` vs `run()`

Create a thread and call:

```
thread.run();  
thread.start();
```

Print current thread name inside `run()` .

Explain the difference.

Program 4: Download Then Process

Create `DownloadThread` .

It should print:

```
Downloading file...  
Download completed
```

Main thread should wait for download thread using `join()` .

After that print:

```
Processing downloaded file
```

Program 5: Order Processing With join()

Create three threads:

- `PaymentTask`
- `InvoiceTask`
- `EmailTask`

Rules:

1. Payment should complete first.
2. Invoice should generate after payment.
3. Email should send after invoice.

Use `join()` to control order.

Program 6: Race Condition Demo

Create `Counter` class with:

- `int count`
- `increment()` method

Create two threads. Each thread should increment count 10000 times.

First run without synchronization.

Then fix using synchronized.

Write:

```
What was the wrong output?  
What changed after synchronized?
```

Program 7: Ticket Booking With synchronized

Create class `TicketBooking` :

- `availableSeats = 1`
- method `bookTicket(String userName)`

Create two user threads:

- Rahul
- Priya

Both should try to book ticket.

Use `synchronized` to avoid overbooking.

Program 8: Ticket Booking With ReentrantLock

Create the same ticket booking example using:

```
ReentrantLock
```

Rules:

- Use `lock()` .
- Use `unlock()` inside `finally` .
- Explain why `finally` is required.

Program 9: Wallet Debit With tryLock()

Create class `Wallet` :

- `balance = 5000`
- method `debit(String userName, double amount)`

Use `tryLock()` .

Rules:

- If lock is available, debit amount if balance is enough.
- If lock is not available, print `"Please try again"` .
- Create two threads trying to debit wallet.

Program 10: FixedThreadPool Email Sender

Create array of 10 email ids.

Use:

```
Executors.newFixedThreadPool(3)
```

Submit email sending tasks.

Print:

```
Sending email to <email> by <thread-name>
```

Program 11: SingleThreadExecutor Audit Log

Use:

```
Executors.newSingleThreadExecutor()
```

Submit these tasks:

```
Order created  
Payment completed  
Invoice generated  
Email sent
```

Explain why `SingleThreadExecutor` is useful here.

Program 12: ScheduledExecutorService

Use `ScheduledExecutorService` to print:

```
Checking payment status
```

after 3 seconds.

Then create another program to print same message every 5 seconds.

Program 13: Callable And Future

Create `MarksCalculator` using `Callable<Integer>` .

It should return total marks of 5 subjects.

Submit using `ExecutorService` and print total using `Future`.

Program 14: Payment Status With Callable

Create `PaymentStatusTask` implements `Callable<String>`.

It should take `orderId` and return:

```
Payment SUCCESS for order <orderId>
```

Use `Future` to get result.

Program 15: Dashboard Loading

Create three Callable tasks:

- `loadProfile`
- `loadOrders`
- `loadWallet`

Each task should sleep for different time and return a message.

Run all using `FixedThreadPool` of 3.

Print all results and then:

```
Dashboard ready
```

Program 16: invokeAll()

Use `invokeAll()` for three Callable tasks:

- total students
- total courses
- total payments

Print all results.

Program 17: invokeAny()

Create three Callable tasks:

- Provider A
- Provider B
- Provider C

Each returns price message.

Use `invokeAny()` and print first successful result.

Part E: Concept Mapping

Complete the table:

Situation	Concept Used
New path of execution	
Method that contains thread task	
Method that creates new thread	
Pause current thread	
Wait for another thread to finish	
Output order can change	
Multiple threads changing same data	
Protect shared data simply	
Lock only small critical section	
Manual advanced locking	
Try lock without waiting forever	
Pool of reusable worker threads	
One worker, ordered execution	
Fixed number of worker threads	
Run task after delay	
Task that returns value	
Future result of task	
Run many tasks and get all results	
Get one successful result	

Part F: Difference Questions

Write differences:

1. Process vs Thread
2. Thread vs Runnable
3. `start()` vs `run()`
4. `sleep()` vs `join()`
5. User thread vs Daemon thread
6. Synchronized method vs Synchronized block
7. `synchronized` vs `ReentrantLock`
8. `lock()` vs `tryLock()`
9. `execute()` vs `submit()`
10. Runnable vs Callable

11. `FixedThreadPool` vs `SingleThreadExecutor`
12. `FixedThreadPool` vs `CachedThreadPool`
13. `shutdown()` vs `shutdownNow()`
14. `invokeAll()` vs `invokeAny()`

Part G: Real-World Thinking Questions

1. In a course enrollment system, why can race condition happen?
2. In ticket booking, why should only one thread book the last seat?
3. In wallet debit, why is locking important?
4. In bulk email sending, why is `ExecutorService` better than raw threads?
5. In audit logging, why can `SingleThreadExecutor` be useful?
6. In dashboard loading, why can multiple `Callable` tasks improve waiting time?
7. In payment status checking, where can `ScheduledExecutorService` be used?
8. In external price provider system, where can `invokeAny()` be useful?
9. Why should `Future.get()` not be called too early after each submit?
10. Why should `unlock()` always be inside `finally` ?

Part H: Mini Project

Create a mini backend-style order processing simulation.

Classes:

```
Order
- orderId
- customerName
- amount

PaymentTask implements Callable<String>
- returns payment status

InvoiceTask implements Callable<String>
- returns invoice status

EmailTask implements Runnable
- sends confirmation email
```

Requirements:

1. Use `ExecutorService`.
2. Payment task should return result using `Future`.
3. Invoice should run after payment success.
4. Email should run after invoice generation.
5. Print thread name in every task.
6. Shutdown `ExecutorService` properly.

Expected output style:

Payment SUCCESS for order 101 by pool-1-thread-1
Invoice generated for order 101 by pool-1-thread-2
Email sent to Rahul by pool-1-thread-3
Order processing completed

Submission Checklist

Before submitting, check:

- Did you use `start()` instead of direct `run()` for thread execution?
- Did you mention when output order is not guaranteed?
- Did you use `join()` where one task must wait?
- Did you protect shared data using `synchronized` or `Lock`?
- Did you use `unlock()` inside `finally`?
- Did you call `shutdown()` on `ExecutorService`?
- Did you use `Callable` only when result is needed?
- Did you explain real-time use after coding questions?
- Did you write expected output before running?

